

Data-Driven Problem Solving

The Cooper Union for the Advancement of Science and Art
ME 371 - Fall 2025

Masoud Masoumi

Agenda

1. Introduction

- What is a decision tree?
- Tree structure and terminology

2. Algorithms

- Top-down recursive approach

3. Building Decision Trees

- Information gain and entropy
- Feature selection criteria

4. Implementation

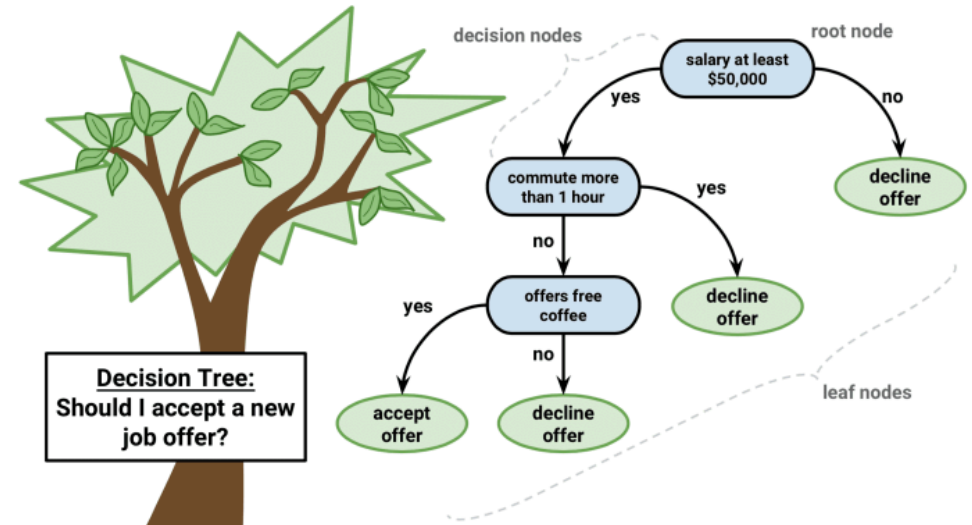
Introduction

What is a Decision Tree?

A decision tree is a **flowchart-like tree structure** used for classification and regression tasks.

Key Components:

- **Internal nodes (decisions):** Each represents a test on a feature
- **Branches:** Represent outcomes of the test
- **Leaf nodes (terminal nodes):** Hold the final class label or value
- **Root node:** The topmost node where the tree begins



Ref for the image: <https://medium.com/analytics-vidhya/mathematics-behind-decision-tree-73ee2ef82164>

Why Decision Trees?

Advantages:

- **Interpretable:** Easy to understand and visualize
- **No feature scaling needed:** Works with raw data
- **Handles both numerical and categorical data**
- **Non-parametric:** Makes no assumptions about data distribution
- **Captures non-linear relationships**
- **Feature importance:** Automatically ranks feature relevance

Disadvantages:

- **Prone to overfitting:** Can create overly complex trees
- **Unstable:** Small data changes can result in different trees
- **Biased toward dominant classes** in imbalanced datasets
- **Greedy algorithm:** May not find the globally optimal tree

The Algorithm

Top-Down Recursive Approach

Most decision tree algorithms follow a top-down, recursive, divide-and-conquer strategy:

Step-by-step Process:

1. Start at the root

- All training examples begin at the root node

2. Select the best feature

- Use a statistical measure (e.g., information gain, Gini impurity)
- Choose the feature that best separates the classes

3. Split the data

- Partition data based on the selected feature
- Create branches for each possible outcome

4. Recurse on subsets

- Repeat the process for each child node
- Continue until stopping criteria are met

Stopping Criteria

When do we stop growing the tree?

The recursive partitioning stops when one of the following conditions is met:

1. **Pure node**

- All samples in a node belong to the same class
- No further split can improve purity

2. **No remaining features**

- All features have been used for splitting
- Cannot split further along the current branch

3. **No samples left**

- A branch has no training samples
- Cannot make further decisions

4. **Hyperparameter constraints** (user-defined)

- Maximum depth reached
- Minimum samples per leaf reached
- Minimum impurity decrease threshold not met

Building A Decision Tree

Information Gain and Entropy

Entropy: Measuring Impurity

Let p_i be the probability that an arbitrary sample in set D belongs to class C_i .

Entropy (expected information needed to classify a sample):

$$Info(D) = - \sum p_i \log_2(p_i)$$

- Entropy = 0: All samples belong to one class (pure)
- Entropy = 1: Samples equally distributed across classes (maximum impurity)

Information after split (using feature A to split D into v partitions):

$$Info_A(D) = \sum \left(\frac{|D_i|}{|D|} \right) \times Info(D_i)$$

Information Gain (reduction in entropy):

$$Gain(A) = Info(D) - Info_A(D)$$

Example: Computing Information Gain

age	income	student	credit_rating	buys_computer
youth	high	no	fair	no
youth	high	no	excellent	no
mid_age	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
mid_age	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
mid_age	medium	no	excellent	yes
mid_age	high	yes	fair	yes
senior	medium	no	excellent	no

$$Info(D) = I(9,5) = -\frac{9}{14} \log_2\left(\frac{9}{14}\right) - \frac{5}{14} \log_2\left(\frac{5}{14}\right) = 0.940$$

$$Info_{age}(D) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.694$$

$\frac{5}{14} I(2,3)$ means “age = youth” has 5 out of 14 samples, with 2 yes’s and 3 no’s.

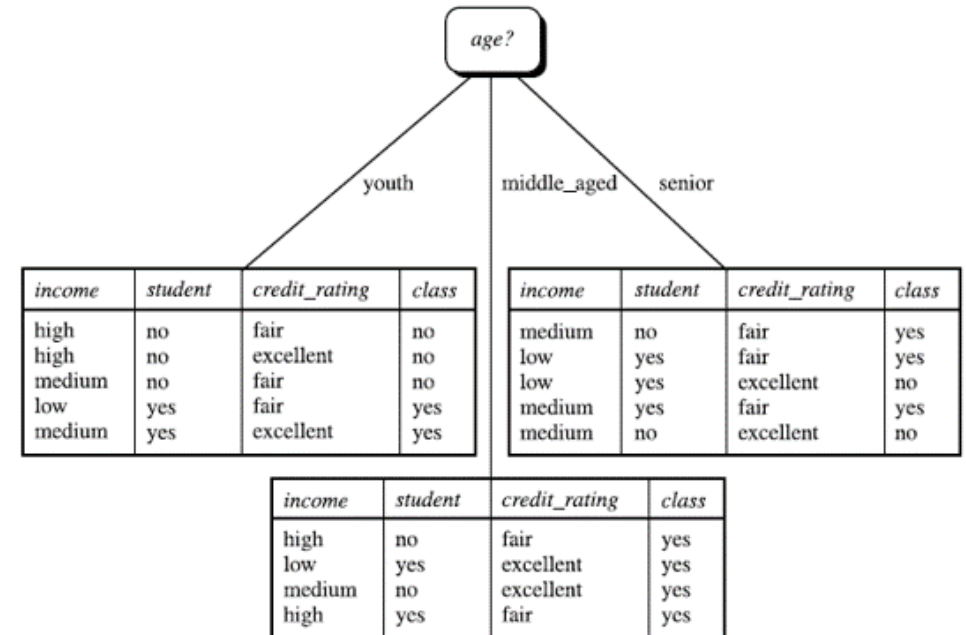
$$\begin{aligned} Info_{age}(D) &= \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) \\ &\quad + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) \\ &\quad + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right) \\ &= 0.694 \text{ bits.} \end{aligned}$$

$$Gain(age) = Info(D) - Info_{age}(D) = 0.246$$

Example: Selecting the Best Feature

Computing gain for all features:

- **Age:** 0.246
- **Income:** 0.029
- **Student:** 0.151
- **Credit Rating:** 0.048

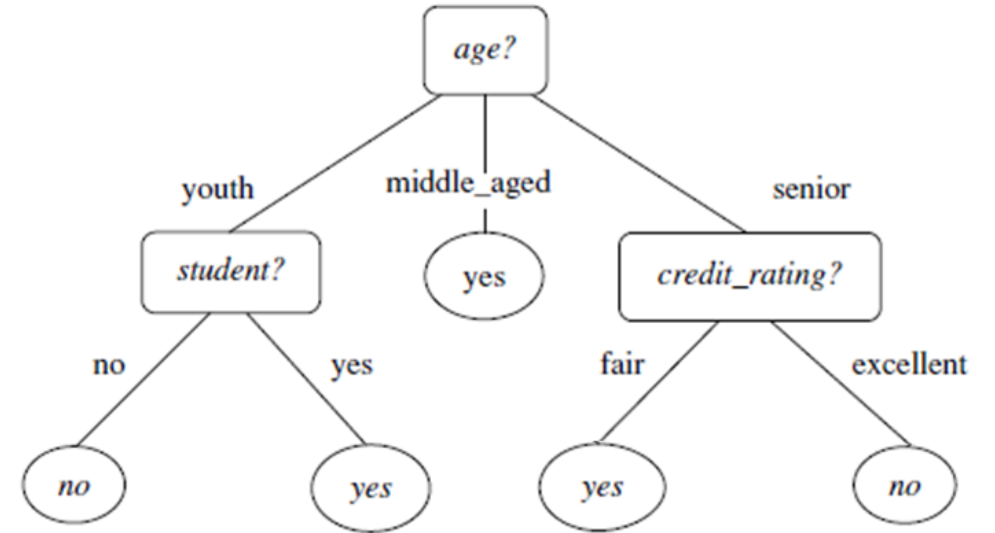


Result: Age has the highest information gain → Use Age as the root node split

Example: Recursive Splitting

Observations:

- Samples in **age = middle_aged** all belong to class "yes"
→ Create a leaf node labeled "yes"
- For **age = youth** and **age = senior**:
→ Repeat the process
→ Calculate entropy and information gain for remaining features
→ Select feature with highest gain



Ref for the flowchart: "Data Mining: Concepts and Techniques" by Han, Pei, and Kamber

Gini Impurity: Alternative to Entropy

Gini Impurity is another measure used:

$$Gini(D) = 1 - \sum p_i^2$$

Interpretation:

- Gini = 0: Pure node (all samples same class)
- Gini = 0.5: Maximum impurity for binary classification
- Lower Gini = Better split

Gini vs. Entropy:

- Gini is computationally faster (no logarithm)
- Both produce similar trees
- Gini tends to isolate the most frequent class
- Entropy tends to produce more balanced trees

Scikit-learn Default: Uses Gini impurity for classification